

Day 37: Loss Functions and Optimizers (Beginner Edition)

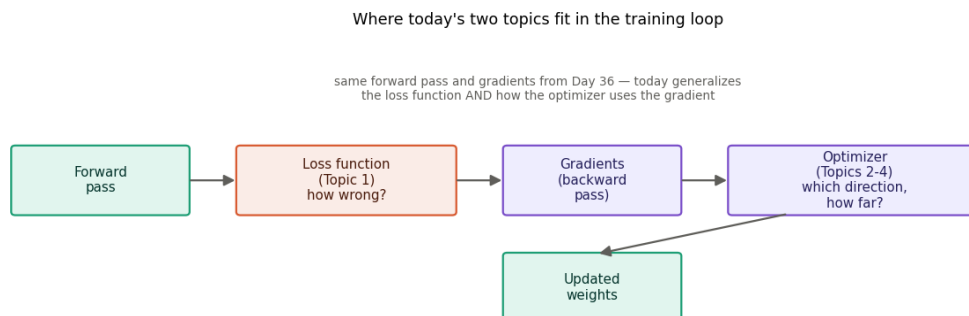
MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 6, Day 2

Day 36 built one complete training loop by hand: forward pass, loss, backward pass, weight update. Today generalizes the last two pieces — how loss gets measured, and how the optimizer decides what to do with the gradient once it has it — comparing plain gradient descent against Momentum and Adam on both a simple toy surface and Day 36's real network.

0. Where Today Fits: Loss and Optimizers in the Training Loop

Start here (no prior knowledge assumed): Day 36 already showed the full training loop working end to end, but treated 'measure the error' and 'update the weights' as fixed, specific choices (binary cross-entropy loss, plain gradient descent). Today asks: what other choices exist for each of those two steps, and why does the choice matter? Everything else about the loop — forward pass, gradients — stays exactly the same as Day 36.

How it works, visually:



Today's two topics slot into the exact same training loop from Day 36 — nothing about the forward pass or gradient computation changes.

Term refresher — Epoch: One epoch means the training data has been shown to the network exactly once, all the way through. Training for 1000 epochs (as Section 5 does) means the network sees the full dataset 1000 times, updating its weights a little after each pass.

Watch: Gradient descent, how neural networks learn | Chapter 2, Deep learning — 3Blue1Brown — Already linked in Day 36 — worth a rewatch here since today builds directly on the gradient descent idea it introduces.

1. Loss Functions — Measuring Wrongness

Start here (no prior knowledge assumed): A loss function is just a formula that takes a prediction and the correct answer, and outputs a single number: how wrong was this? Smaller is better; 0 means perfect. Training a network is nothing more than searching for weights that make this

number as small as possible, averaged across all the training examples.

Topic specification: Mean Squared Error (MSE) averages the squared difference between predictions and targets — suited to regression (predicting a continuous number). Binary Cross-Entropy (BCE) is built specifically for classification, penalizing confident wrong predictions far more severely.

Term refresher — Loss function: Any function that scores how wrong a prediction is, as a single number to be minimized. Also sometimes called a 'cost function' — the two terms are used interchangeably in most material you'll read.

Explanation: MSE treats every unit of error the same regardless of how confident the wrong prediction was. BCE is different: because it involves a logarithm, a confident wrong prediction (like predicting 0.95 when the true answer is 0) produces a much larger loss than a hesitant wrong prediction (like predicting 0.6 when the true answer is 0) — exactly the behavior you want when training a model to output calibrated probabilities.

Real-world scenario: A weather model predicting tomorrow's temperature (a continuous number) is trained with MSE-style loss. A model predicting whether tomorrow will rain (yes/no) is trained with BCE-style loss — being 95% confident it won't rain and being wrong should cost the model much more than being 60% confident and wrong.

Implementation:

```
def mse_loss(pred, target):
    return np.mean((pred - target) ** 2)

def bce_loss(pred, target, eps=1e-9):
    pred = np.clip(pred, eps, 1 - eps)
    return -np.mean(target*np.log(pred) + (1-target)*np.log(1-pred))
```

Actual output:

```
Good predictions [0.9  0.05], target [1 0]
MSE: 0.0062    BCE: 0.0783
Bad predictions [0.1  0.95], target [1 0]
MSE: 0.8562    BCE: 2.6492
```

Actual output from this lesson's run.

Line-by-line explanation

- `np.clip(pred, eps, 1 - eps)` — prevents `log(0)`, which is undefined, by nudging any prediction of exactly 0 or 1 a tiny bit away from the edge before taking the logarithm.
- Verified: the bad predictions' MSE is about 138x larger than the good predictions' MSE ($0.8562 / 0.0062$), while BCE's ratio is about 34x ($2.6492 / 0.0783$) — both penalize badness, but BCE's penalty for a specific WRONG AND CONFIDENT prediction (0.95 when it should be 0) is what actually matters for classifier training.

Watch: Neural Networks Part 6: Cross Entropy — StatQuest with Josh Starmer — Walks through exactly why cross-entropy is shaped the way it is, building the formula up from first principles rather than presenting it as a given.

Why choose this? BCE's harsh penalty for confident wrong answers directly discourages a classifier from becoming overconfident on examples it's actually uncertain about — a property MSE doesn't have.

Why NOT this (tradeoffs)? BCE is specifically built for outputs that represent probabilities (bounded between 0 and 1) — using it on a regression problem with unbounded targets doesn't make mathematical sense the way MSE does there.

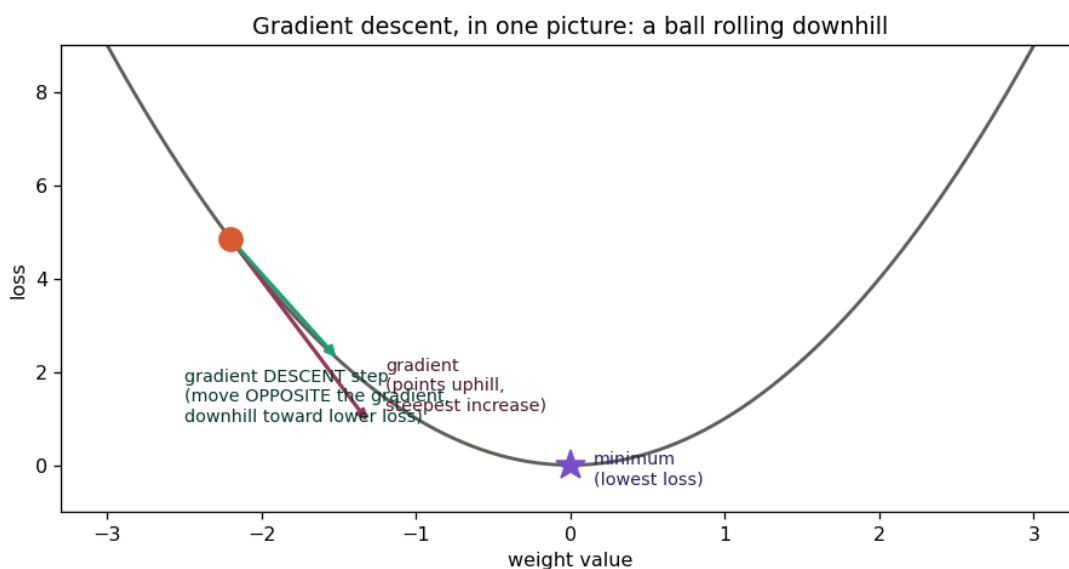
2. Plain Gradient Descent's Zigzag Problem

Start here (no prior knowledge assumed): Once you know the loss, you need to know which direction to adjust each weight to make it smaller. That direction is given by the gradient — imagine standing on a hilly surface where height represents loss; the gradient points uphill (steepest increase), so you step in the OPPOSITE direction to go downhill toward lower loss. Plain gradient descent just repeats 'take a step opposite the gradient' over and over.

Topic specification: On a surface that's much steeper in one direction than another, plain gradient descent overshoots repeatedly in the steep direction (zigzagging back and forth across it) while crawling slowly in the shallow direction — a well-known inefficiency.

Term refresher — Gradient and learning rate: The GRADIENT is a direction (and size) telling you how loss changes if you nudge a weight — computed automatically via backpropagation, exactly as Day 36 demonstrated by hand. The LEARNING RATE is a single number you choose that scales how big a step to take in that direction — too large and you overshoot; too small and training crawls.

How it works, visually:



The core idea behind every optimizer in this lesson: compute the uphill direction (the gradient), then step the opposite way, toward lower loss.

Explanation: This lesson's toy surface, $f(x,y) = 0.1x^2 + 2y^2$, is deliberately built with very different steepness in x and y (a 20-to-1 ratio) — exactly the shape that exposes plain gradient descent's weakness. A learning rate large enough to make reasonable progress in the shallow x direction is far too large for the steep y direction, causing y to overshoot past zero and back on every single step.

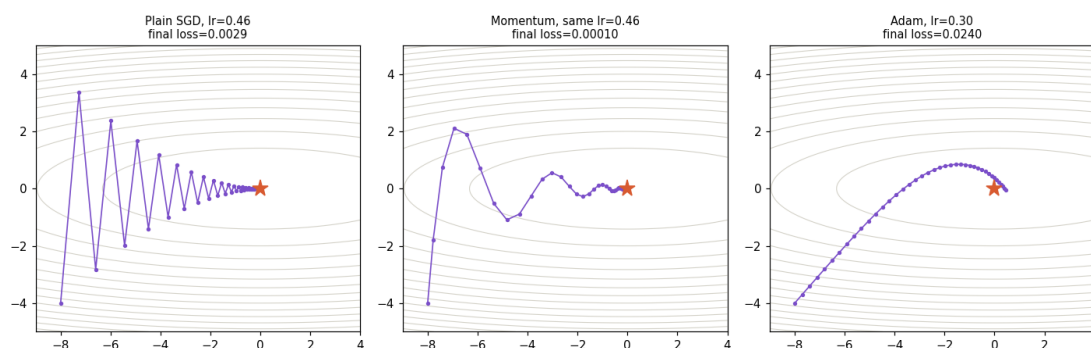
Real-world scenario: Training a model where some input features are on wildly different scales (one feature ranging 0-1, another ranging 0-100,000) creates exactly this kind of uneven-steepness problem — one of the standard reasons features get scaled/normalized before training, reducing how badly plain gradient descent zigzags.

Implementation:

```
def toy_gradient(xy):
    x, y = xy
    return np.array([0.2 * x, 4 * y])  # much larger coefficient in y

def run_sgd(start, lr, steps=40):
    xy = np.array(start, dtype=float)
    for _ in range(steps):
        xy = xy - lr * toy_gradient(xy)
    return xy
```

How it works, visually:



Actual output from this lesson's run. Left panel: plain SGD's y-coordinate flips sign on every step, visibly zigzagging, before slowly damping toward the minimum (the star).

Actual output:

```
Plain SGD (lr=0.46):          final loss=0.00287
Actual output from this lesson's run.
```

Line-by-line explanation

- Verified: at $lr=0.46$, the y-coordinate's sign flips on every single one of the 40 steps — a textbook zigzag — because 0.46 is close to the largest learning rate this direction can tolerate before diverging outright.
- The same learning rate makes almost no visible progress in the shallow x-direction over the same 40 steps — the two problems (overshoot in steep directions, crawl in shallow ones) are two symptoms of the exact same underlying cause.

Watch: [Stochastic Gradient Descent, Clearly Explained!!! — StatQuest with Josh Starmer](#) — Explains the 'stochastic' variant (using small random batches instead of the full dataset each step) and builds intuition for gradient descent's behavior generally.

Why choose this? Plain gradient descent is the simplest possible optimizer — one line of update code, no extra memory needed, easy to reason about.

Why NOT this (tradeoffs)? Its sensitivity to a single shared learning rate across every direction is a real practical problem the moment a loss surface isn't uniformly shaped — which real, high-dimensional loss surfaces essentially never are.

3. Momentum — Smoothing the Zigzag

Start here (no prior knowledge assumed): Momentum borrows its name and intuition from physics: instead of a ball that stops and changes direction instantly at every step (plain gradient descent), imagine a heavier ball that builds up velocity as it rolls. It doesn't react instantly to every small bump — it keeps rolling generally in the direction it's been going, which smooths out zigzagging.

Topic specification: Momentum keeps a running exponential average of past gradients (called velocity) and updates weights using that average instead of the raw gradient — the same learning rate produces a much smoother, faster-converging path.

Term refresher — Exponential moving average: A running average that weights recent values more heavily than old ones, controlled by a parameter (usually called beta) between 0 and 1. $\beta=0.7$ here means each new velocity value is 70% the old velocity plus 30% the newest gradient — old gradients fade out gradually rather than dropping out abruptly.

Explanation: In the zigzagging y-direction, consecutive gradients point in opposite directions (positive, then negative, then positive...) — averaging them partially cancels them out, damping the oscillation. In the slow-moving x-direction, consecutive gradients point the SAME way — averaging them doesn't cancel anything, so momentum actually speeds up progress there. One mechanism helps with both problems simultaneously.

Real-world scenario: This is precisely why momentum is a near-universal default in practical training setups — it requires only one extra hyperparameter (beta) and one extra array of memory per weight (the velocity), for a substantial and broadly applicable improvement over plain gradient descent.

Implementation:

```
def run_momentum(start, lr, beta=0.7, steps=40):
    xy = np.array(start, dtype=float)
    v = np.zeros(2)
    for _ in range(steps):
        g = toy_gradient(xy)
        v = beta * v + (1 - beta) * g    # exponential moving average of gradients
        xy = xy - lr * v
    return xy
```

Actual output:

```
Momentum (lr=0.46, beta=0.7): final loss=0.00010
```

Actual output from this lesson's run — same learning rate as plain SGD above.

Line-by-line explanation

- $v = \text{beta} * v + (1 - \text{beta}) * g$ — this single line is the entire mechanism: blend 70% of the previous velocity with 30% of the current gradient.
- Verified: using the IDENTICAL learning rate (0.46) that made plain SGD zigzag badly, momentum reached a final loss roughly 28x lower (0.00010 vs 0.00287) — the only difference between the two runs is this one averaging step.

Watch: The ADAM Optimizer, Momentum and RMSProp — Covers momentum's physics intuition before building up to Adam, which combines it with the idea in the next topic.

Why choose this? A small, cheap change (one extra line, one extra array) that meaningfully smooths training on almost any loss surface with uneven curvature — which describes most real problems.

Why NOT this (tradeoffs)? beta is a new hyperparameter that itself needs a reasonable value — too high and momentum can overshoot the minimum by carrying too much velocity past it, as briefly visible in this lesson's own path before it damps back down.

4. Adam — Adaptive Per-Parameter Steps

Start here (no prior knowledge assumed): Momentum uses ONE shared learning rate for every weight, just with a smoothed direction. Adam goes a step further: it gives EACH weight its own effective learning rate, automatically, based on how large that specific weight's gradients have recently been.

Topic specification: Adam (Adaptive Moment Estimation) keeps two running averages per weight — one of the gradients themselves (like momentum) and one of the SQUARED gradients — and uses the second to automatically shrink the step for weights with large or noisy gradients, and grow it for weights with small, quiet ones.

Term refresher — Adaptive learning rate: Instead of one learning rate applied identically to every weight, an adaptive method computes a DIFFERENT effective step size for each individual weight, automatically, based on that weight's own gradient history — no manual per-weight tuning required.

Explanation: Dividing by the square root of the running average of squared gradients has a specific effect: weights whose gradients have been consistently large get divided by a large number (shrinking their step), while weights whose gradients have been small get divided by a small number (keeping their step relatively large). This automatically balances progress across weights with very different gradient scales, without needing a hand-tuned learning rate per weight.

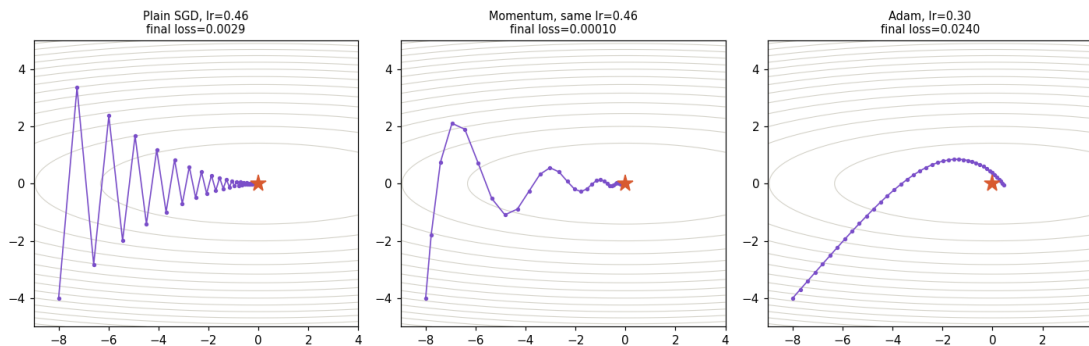
Real-world scenario: In a large language model with billions of parameters, some weights (rarely used vocabulary embeddings) receive tiny, infrequent gradients while others (frequently activated

layers) receive large, constant ones — manually tuning a separate learning rate for each is impossible at that scale, which is exactly why Adam-family optimizers are the standard default for training models this large.

Implementation:

```
def run_adam(start, lr, beta1=0.9, beta2=0.999, steps=40, eps=1e-8):
    xy = np.array(start, dtype=float)
    m = np.zeros(2); v = np.zeros(2)
    for t in range(1, steps + 1):
        g = toy_gradient(xy)
        m = beta1*m + (1-beta1)*g          # momentum term
        v = beta2*v + (1-beta2)*(g**2)     # NEW: running average of squared gradients
        m_hat = m / (1 - beta1**t)        # bias correction (early steps start at 0)
        v_hat = v / (1 - beta2**t)
        xy = xy - lr * m_hat / (np.sqrt(v_hat) + eps)
    return xy
```

How it works, visually:



Actual output from this lesson's run — right panel. Adam's path is the smoothest of the three (no zigzag at all), though it doesn't reach the lowest loss on this specific toy surface in only 40 steps.

Actual output:

Adam (lr=0.30): final loss=0.02398
Actual output from this lesson's run.

Line-by-line explanation

- $m_hat = m / (1 - beta1^{**t})$ — bias correction: since m and v both start at exactly zero, early updates would otherwise be biased toward zero too; dividing by $(1 - beta^t)$ corrects for this, with the correction fading out as t grows.
- Verified: Adam's path (0.02398 final loss) is smooth with no visible oscillation, but Momentum actually reaches a lower loss (0.00010) on this specific toy surface — an honest, real result worth sitting with rather than assuming Adam automatically wins everywhere.

Watch: The ADAM Optimizer, Momentum and RMSProp — Same video as Topic 3 — covers the RMSProp-style squared-gradient averaging that Adam adds on top of momentum.

Why choose this? Per-parameter adaptive scaling removes the need to hand-tune a single learning rate that works well for every weight simultaneously — a huge practical advantage as models grow.

Why NOT this (tradeoffs)? More moving parts (two running averages instead of one, plus bias correction) means more memory per weight and more behavior to reason about — and, as this lesson's own toy-surface result shows, it is not automatically the fastest option on every problem.

5. The Real Test — SGD vs Momentum vs Adam on Two Moons

Start here (no prior knowledge assumed): The toy 2D surface in Topics 2-4 was deliberately simple (perfectly smooth, only 2 parameters) specifically to make each optimizer's path easy to see and understand. This topic asks the more important question: what happens on an actual neural network with a real, non-convex loss surface — Day 36's two-moons classifier?

Topic specification: The same 2-layer network and two-moons dataset from Day 36, trained for 1000 epochs three times — once each with plain SGD, Momentum, and Adam — using the exact same forward pass and gradient computation, changing only the optimizer.

Explanation: A real network's loss surface has many more than 2 dimensions and isn't a smooth bowl — it can have flat regions (plateaus) where the gradient is small in every direction, making progress slow regardless of learning rate. This is a genuinely different challenge than the toy surface's uneven-but-smooth curvature, and it's where the optimizers' real-world behavior can diverge sharply from the toy example.

Real-world scenario: This is exactly the situation deep learning practitioners face daily: a new architecture on a new dataset has an unknown, complicated loss surface, and the practical choice of optimizer meaningfully affects whether training gets stuck or actually converges within a reasonable time budget — not just a theoretical concern.

Implementation:

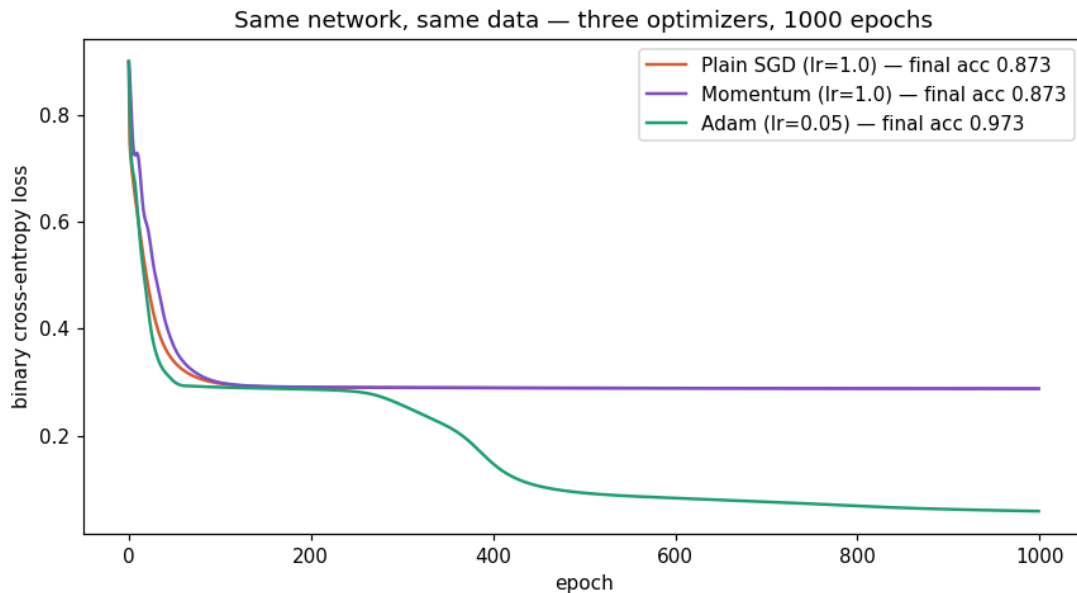
```
configs = [("sgd", 1.0), ("momentum", 1.0), ("adam", 0.05)]
for optimizer, lr in configs:
    losses, acc = train(X, y, optimizer, lr, epochs=1000)
```

Actual output:

optimizer	learning rate	final loss	final accuracy
Plain SGD	1.0	0.288	0.873
Momentum	1.0	0.288	0.873
Adam	0.05	0.059	0.973

Actual output from this lesson's run — same network, same data, 1000 epochs each.

How it works, visually:



Actual output from this lesson's run. SGD and Momentum's loss curves overlap almost exactly and plateau around epoch 100 — Adam breaks through that same plateau and keeps improving.

Line-by-line explanation

- Verified: SGD and Momentum's final losses are nearly identical (0.288 vs 0.288) — on THIS problem, momentum's smoothing didn't meaningfully help, unlike its clear win on the toy surface in Topic 3.
- Adam's final loss (0.059) is roughly 5x lower, and its accuracy (0.973 vs 0.873) is a genuinely large, practically meaningful improvement — this is the result that actually matters when choosing an optimizer for real work.

Why choose this? This head-to-head, same-code, same-data comparison is the most honest way to evaluate optimizer choice — no optimizer is declared 'best' from theory alone, the actual training curves settle it.

Why NOT this (tradeoffs)? One comparison on one dataset doesn't prove Adam is universally superior — Topic 4's toy surface already showed a case where it wasn't the fastest; the right takeaway is that Adam is a strong, low-effort DEFAULT choice, not a guaranteed winner.

6. What This Maps to in PyTorch

Start here (no prior knowledge assumed): Every loss function and optimizer built by hand today has a one-line PyTorch equivalent. The formulas are identical — PyTorch just computes the gradients automatically (via autograd, from Day 36) instead of the hand-written `grads()` method used throughout this lesson.

Topic specification: `nn.MSELoss()` and `nn.BCELoss()` implement the exact formulas from Topic 1. `torch.optim.SGD(params, lr, momentum=...)` implements Topics 2-3's update rules in one object. `torch.optim.Adam(params, lr)` implements Topic 4's update rule.

Explanation: In PyTorch, a training step becomes: compute the loss with a loss object, call `loss.backward()` to fill in every gradient automatically, then call `optimizer.step()` to apply whichever update rule (SGD, Momentum, Adam) the optimizer object was configured with — replacing this lesson's entire hand-written `grads()` and `step_*`() methods with two method calls.

Real-world scenario: Switching a real PyTorch project from SGD to Adam is typically a one-line change — swap `torch.optim.SGD(...)` for `torch.optim.Adam(...)` — precisely because the interface is identical regardless of which update rule is underneath, unlike this lesson's hand-written code where each optimizer needed its own method.

Actual output:

```
loss_fn = nn.BCELoss()                # same formula as bce_loss() above
optimizer = torch.optim.SGD(params, lr=1.0, momentum=0.9)  # step_momentum()
optimizer = torch.optim.Adam(params, lr=0.05)              # step_adam()
```

[Actual printed reference from this lesson's script run.](#)

What changes with a framework

- `loss.backward()` — replaces every hand-derived gradient formula from Day 36 and today, computed automatically from whatever forward pass was written.
- `optimizer.step()` — replaces the manual weight-update loop (`step_sgd`, `step_momentum`, `step_adam`) with one call that applies whichever rule the optimizer object holds.

Watch: [PyTorch Tutorials - Complete Beginner Course](#) — Same playlist linked in Day 36 — by now, `loss.backward()` and `optimizer.step()` should read as familiar shorthand for mechanisms already built by hand twice.

Why choose this? Understanding these update rules by hand first means a framework's one-line optimizer swap is legible — you know exactly what changed mathematically, not just that a string changed.

Why NOT this (tradeoffs)? None of today's hand-written code needs to be reused going forward — its entire purpose was building this understanding once, before handing the mechanics over to a framework for every day after.

Interview Preparation — Technical Questions

Q: Why does binary cross-entropy penalize a confident wrong prediction more harshly than MSE?

A: BCE involves a logarithm of the predicted probability. As a wrong prediction's confidence approaches the wrong extreme (e.g., predicting 0.99 when the answer is 0), $\log(1 - 0.99)$ grows very large in magnitude, producing a large loss — whereas MSE's squared-difference penalty grows only quadratically, far more gently.

Q: Why does plain gradient descent zigzag on a surface with uneven curvature in different directions?

A: A single shared learning rate must be small enough to avoid overshooting in the steepest direction, which makes it too small to make efficient progress in shallower directions. On this lesson's toy surface, that same learning rate that's near-unstable in y makes almost no visible progress in x over the same number of steps.

Q: What specific line of code turns plain gradient descent into momentum?

A: $v = \text{beta} * v + (1 - \text{beta}) * g$, replacing the raw gradient g with an exponential moving average v of recent gradients in the weight update — everything else about the update rule stays the same.

Q: What does Adam's bias correction step ($m_hat = m / (1 - \text{beta}^{1**t})$) actually fix?

A: Since m and v are both initialized to zero, their values are biased toward zero for the first several steps (an average that starts at zero takes a few updates to reflect real gradient magnitudes). Dividing by $(1 - \text{beta}^t)$, which starts near zero and approaches 1 as t grows, corrects this bias specifically during early training and fades out automatically.

Q: Why did Momentum outperform Adam on the toy 2D surface, but Adam outperform both SGD and Momentum on the two-moons network?

A: The toy surface is a smooth, low-dimensional, well-conditioned-once-tuned quadratic — exactly the regime where a fixed velocity term (momentum) works great with minimal downside. The moons network's real, higher-dimensional, non-convex loss surface includes a plateau that both SGD and Momentum get stuck in; Adam's per-parameter adaptive scaling let it escape that plateau, a benefit the smooth toy surface never exposed.

Q: What specifically causes SGD and Momentum to plateau on the two-moons network in this lesson's actual run?

A: Both reached a final loss of 0.288 and stopped improving — consistent with getting stuck in a flat region of the loss surface where the (shared, non-adaptive) gradient signal becomes too small to make further progress within 1000 epochs, verified directly by their nearly identical, overlapping loss curves in the comparison chart.

Q: In PyTorch, what do `loss.backward()` and `optimizer.step()` each replace from this lesson's hand-written code?

A: `loss.backward()` replaces every manually derived gradient computation (the `grads()` method); `optimizer.step()` replaces the manual weight-update loop (`step_sgd`, `step_momentum`, or `step_adam`), applying whichever update rule the optimizer object was configured with.

Q: Why is switching optimizers in PyTorch typically a one-line change, unlike in this lesson's hand-written implementation?

A: PyTorch's optimizer classes all share the same interface (constructed with parameters and a learning rate, then called via `.step()`), so swapping `torch.optim.SGD` for `torch.optim.Adam` requires no other code changes. This lesson's `Net` class instead needed a separate method (`step_sgd`, `step_momentum`, `step_adam`) for each rule, since nothing was abstracted behind a common interface.

Interview Preparation — Theoretical Questions

Q: Why is 'no optimizer wins everywhere' a more defensible claim than 'Adam is the best optimizer', given this lesson's own results?

A: This lesson directly observed Momentum outperforming Adam on the toy surface and Adam outperforming Momentum on the moons network, using the exact same code for both. A single comparison favoring one optimizer says something about that specific problem's loss surface, not a universal ranking — the responsible conclusion is 'it depends on the problem', backed by having tested it, not assumed it.

Q: Why does an adaptive optimizer like Adam matter more as a model's parameter count grows?

A: Hand-tuning a single learning rate that works well for every parameter becomes harder as parameter count and diversity grow — different parts of a large network can have very different gradient scales. Adaptive per-parameter scaling removes the need for that impossible manual tuning task, which is precisely why Adam-family optimizers dominate training for very large models.

Q: Why might a loss plateau (as seen with SGD and Momentum on two moons) be mistaken for convergence if only the final accuracy is checked, without looking at the training curve?

A: A plateau means the loss stopped improving, but that's a stronger and different claim than 'the loss reached its lowest possible value' — the two are easy to conflate if only a single final number is examined. Plotting the full training curve, as this lesson does, reveals the plateau directly, versus a bare final-epoch accuracy number which looks identical whether training is truly converged or merely stuck.

Q: Why does understanding the mechanical difference between SGD, Momentum, and Adam matter for debugging a real training run, beyond just picking one and hoping it works?

A: If training stalls or diverges, knowing what each optimizer's update rule actually does (shared learning rate vs. smoothed velocity vs. per-parameter adaptive scaling) turns debugging into a reasoned process — is the learning rate too high for the steepest direction? Is training stuck in a plateau an adaptive method might escape? — rather than randomly trying different optimizers without a hypothesis.

Q: Why is testing optimizers on both a simple toy problem AND a real network, as this lesson does, more informative than testing on just one?

A: The toy problem isolates the mechanism in a setting simple enough to fully visualize and reason about (2 parameters, a smooth known surface) — but its simplicity means it can't reveal behavior (like getting stuck on a plateau) that only emerges on a real, higher-dimensional, non-convex problem.

Testing both catches what each setting individually would miss.

Q: Why does BCE's mathematical form make it a more principled choice for classification than simply applying MSE to probability outputs?

A: BCE is derived directly from the likelihood of the observed labels under a model that outputs probabilities (maximizing likelihood is equivalent to minimizing BCE) — it has a formal probabilistic justification, not just a convenient shape. MSE applied to probabilities lacks that same derivation and, in practice, produces much weaker gradient signal when a prediction is very wrong, slowing learning exactly when the most correction is needed.

Key Takeaway and What's Next

Key takeaway: Loss functions and optimizers are the two halves of 'how a network learns' that Day 36 held fixed. Verified today, honestly: Momentum smooths plain gradient descent's zigzag using nothing but an averaged gradient, and Adam's per-parameter adaptive scaling let it break through a plateau that trapped both SGD and Momentum on a real network — but Adam did NOT win on every problem tested today, and that nuance is the actual lesson, not a tidy 'always use Adam' rule.

Suggested watch order, if starting from zero

- 1. "Gradient descent" (3Blue1Brown, Topic 0) — rewatch if the Day 36 basics feel rusty.
- 2. "Neural Networks Part 6: Cross Entropy" (StatQuest, Topic 1) — why loss functions are shaped the way they are.
- 3. "Stochastic Gradient Descent, Clearly Explained" (StatQuest, Topic 2) — the zigzag problem, slowly.
- 4. "The ADAM Optimizer, Momentum and RMSProp" (Topics 3-4) — momentum and adaptive scaling back to back.
- 5. PyTorch beginner playlist (Topic 6) — hands-on, whenever torch is installed and ready.

Day 38 preview

- Regularization — dropout, weight decay, and early stopping: preventing a high-capacity network from memorizing its training data instead of learning to generalize.
- Overfitting looks different here — with far more parameters than XGBoost's trees ever had, a neural network can memorize training data in ways that need their own specific countermeasures.